ASG Data Platform — Data Team Guide

Everything in one place: what dbt is, how this project is laid out, and how to use dbt Studio.

Written for data engineers and data analysts. No prior dbt experience assumed.

> **Current version.** This guide reflects the latest dbt Studio, which now has **sign-in and role-based access control**, a **Settings** screen for managing users, roles and dataset access, a **unified Documentation page** (three sources × three description engines, plus dbt source declaration), and the ability to **create views and tables from the Workbench**. Sections 11–15 and the cost analysis are new. If you used an earlier build, the biggest change is that you now log in, and what you can do depends on your role.

Contents

1. What dbt is
2. dbt Cloud vs dbt Core
3. How this project is built
4. Getting started
5. The UI, page by page
6. The documentation engines
7. What is inside the generated documentation
8. Guardrails
9. Troubleshooting
10. Reference
11. Signing in and roles (RBAC)
12. The Settings page
13. The unified Documentation page
14. Creating views and tables from the Workbench
15. Documenting tables dbt did not build
16. What it costs

---

1. What dbt is

dbt (data build tool) is how you turn SQL `SELECT` statements into a managed, tested, documented set of tables and views in your warehouse.

The core idea: **you write a SELECT, dbt handles the rest.** You never write `CREATE TABLE`, `DROP`, or `INSERT`. You write one `.sql` file per table containing only a query, and dbt works out the DDL, the dependency order, and the rebuild.

The four things dbt gives you

1. Dependency management through `ref()`

Instead of hardcoding a table name:

```
-- without dbt: brittle, environment-specific
select * from `data-analytics-asg`.`silver_dbt`.`stg__ticket_trans`
```

you write:

```
-- with dbt: portable, and dbt now knows this model depends on that one
select * from {{ ref('stg__ticket_trans') }}
```

dbt reads every `ref()` in the project, builds a dependency graph (a DAG), and runs your models in the correct order automatically. Change the target from dev to prod and every `ref()` repoints itself — the SQL never changes.

2. Testing

You declare expectations in YAML and dbt turns them into SQL that must return zero rows:

```
columns:
  - name: gl_entry_key
    data_tests:
      - unique
      - not_null
```

If a duplicate ever appears, the build fails instead of quietly double-counting downstream.

3. Documentation as code

Descriptions and column types live in YAML next to the SQL, in version control, reviewed in the same pull request as the logic. They cannot drift out of date the way a wiki page does.

4. Environments

The same code runs against a developer sandbox or against production, decided by a target at run time. Nobody edits SQL to promote a change.

What dbt does *not* do

- **It does not move data.** dbt transforms data that is already in BigQuery. If data needs to land in BigQuery first, that is a different tool (Debezium, Fivetran, an external table over Sheets, a manual load).
- **It does not schedule itself.** Something has to invoke `dbt build` on a cadence — an orchestrator, Cloud Scheduler, a CI pipeline.
- **It is not a BI tool.** It prepares the tables your BI layer reads.

---

2. dbt Cloud vs dbt Core

Same transformation engine underneath. The difference is everything around it.

	dbt Cloud (what we used)	dbt Core (what we use now)
Cost	Per-seat subscription	Free, open source
Where it runs	Google's/dbt Labs' servers	Your machine or your orchestrator
Interface	Web IDE, scheduler, docs site, logs	Command line
Scheduling	Built in	You provide it
Credentials	Managed in the Cloud UI	`profiles.yml`

The migration saves the subscription cost. What you lose is the interface: with dbt Core alone, everything happens through terminal commands.

dbt Studio — the UI in `dbt_ui/` — exists to replace that lost interface, and adds the documentation and profiling features dbt Cloud never had.

---

3. How this project is built

The medallion architecture

Data flows through three layers, each with one job. Never skip a layer, and never put one layer's job in another.

```
seeds/CSV ──► BRONZE ──► SILVER ──► GOLD ──► BI
              raw        cleaned     business
```

Bronze — raw landing zone. One row in, one row out. No filtering, no deduplication, no business logic. Light type casting and audit columns only.

Why it matters: because bronze is a faithful copy, you can **replay history** after a business rule changes. If bronze filtered or reshaped anything, that original data is gone forever.

Silver — cleaned and conformed. Deduplication, null handling, trimming, type discipline, derived categories, and quality flags. Rows are **never dropped** for quality reasons — problems are flagged on the row so they stay visible.

Gold — business-facing. Aggregates and facts. The grain is explicit and stable. Measures are ready for BI without further work.

Where each layer physically lands

`macros/generate_schema_name.sql` routes writes based on the target, so a developer cannot overwrite a production table:

Target	bronze goes to	silver goes to	gold goes to	Use
`dev`	`dbt_dev_bronze`	`dbt_dev_silver`	`dbt_dev_gold`	daily work
`test`	`dbt_ci_bronze`	`dbt_ci_silver`	`dbt_ci_gold`	CI
`prod`	`bronze_dbt`	`silver_dbt`	`gold_dbt`	orchestrator only

`prod` writes the exact dataset names the business already reads. Every other target writes to a prefixed sandbox.

> The gold column is listed for completeness, but **dbt Studio never reads or writes gold on any target.** Gold is built by the orchestrator from the command line. See Guardrails.

File layout

```
dbt/
  dbt_project.yml          project config, layer materializations, seed types
  profiles.yml             BigQuery connections (oauth, no secrets — safe to commit)
  packages.yml             dbt_utils 1.4.1, codegen 0.14.1

  models/
    bronze/
      bronze_gl_entries.sql        the model (a SELECT)
      _bronze__models.yml          its documentation and tests
    silver/
      silver_gl_entries.sql
      _silver__models.yml
    gold/
      gold_gl_monthly_summary.sql
      _gold__models.yml
    examples/                      starter smoke-test models

  seeds/
    gl_entries.csv                 synthetic GL data, safe to run anywhere
    _seeds__gl_entries.yml         its column types and tests

  macros/
    generate_schema_name.sql       target-aware dataset routing
    asg_helpers.sql                surrogate keys, audit columns, money casts

  dbt_ui/                          the UI (dbt never parses this folder)
  target/                          generated artifacts — do not edit
```

The convention for each layer

Observed from the existing dbt Cloud output and preserved here:

	Bronze	Silver	Gold
Materialization	table	view	table
Naming	`bronze_*`	`stg__*` / `silver_*`	`fact_*`, `kpi_*`, `dim_*`
Audit column	`_bronze_loaded_at`	`_silver_loaded_at`	`_gold_loaded_at`
Quality flags	none	`_is_*`, `_has_*`	rolled-up counters

---

4. Getting started

One-time setup

1. Authenticate to BigQuery

```
gcloud auth application-default login
```

This expires periodically. When the connection pill in the UI turns red saying "*Reauthentication is needed*", run it again and **restart the server**.

2. Install dbt packages

```
cd C:\Users\ryunu\Documents\work\dbt
dbt deps --profiles-dir .
```

3. Optional — AI documentation

```
pip install google-genai
```

Then get a free key at aistudio.google.com/apikey and paste it into the UI. A personal Google account is fine; see section 6.

Starting the UI

```
cd C:\Users\ryunu\Documents\work\dbt
python dbt_ui\serve.py
```

Opens `http://localhost:8777`. Or double-click `dbt_ui\start_dbt_ui.bat`.

You now sign in. On first start, three default accounts exist — sign in as `manager@gmail.com` / `manager123` for full access, and change the password afterward. See section 11.

Important: only run one server at a time. If the port is taken, it silently moves to the next free one and your browser keeps talking to the old process, so your changes appear not to work. Check with:

```
Get-Process python          # should show one entry
taskkill /F /IM python.exe  # clears all of them
```

Verifying without the UI

```
python dbt_ui\serve.py --check    # environment report, then exits
dbt debug --profiles-dir .        # tests the BigQuery connection
dbt build --profiles-dir . --target dev   # full pipeline: seed, run, test
```

A healthy `dbt build` ends with `PASS=46 WARN=0 ERROR=0 SKIP=0`.

> The command may print a `quota project` warning from Google's auth library and report exit code 1 even on success. It is harmless. To silence it: `gcloud auth application-default set-quota-project data-analytics-asg`

---

5. The UI, page by page

Click a page in the sidebar to switch. The header is always visible.

> **Two things changed recently.** (1) You sign in first — see section 11. (2) There is now a **Settings** page in the sidebar (section 12), and the header no longer has a target dropdown or number-key shortcuts. The page descriptions below still hold; the Documentation page in particular was rebuilt into one unified screen, covered in section 13.

The header

Control	What it does
Target dropdown	Switches `dev` / `test` / `prod`. Drives everything: queries, profiling, dbt commands. Selecting `prod` asks for confirmation twice.
Refresh manifest	Runs `dbt parse`, rebuilding `target/manifest.json`. **Click this after editing any `.sql` or `.yml` file**, otherwise the UI shows stale information.
Connection pill	Green = BigQuery reachable. Red = credentials expired. Click to retest.
Dataset scope (sidebar)	The datasets this instance is allowed to read. See section 8.

Page 1 — Overview

Purpose: project health in one screen.

Shows: model and test counts; documentation coverage; typed-column coverage; the medallion layer breakdown; the result of the last dbt run with its slowest nodes; a health panel listing models missing a description or tests.

Use it when:

- Starting the day: *did last night's build pass?*
- Before a pull request: *is coverage still 100%?*
- Onboarding someone: the whole project's shape on one screen.

Page 2 — Pipeline

Purpose: the medallion architecture as a visual board.

Four columns — Seed, Bronze, Silver, Gold — with each model as a card showing its materialization, column count, test count, and whether it is documented. Cards outside the permitted dataset scope are dimmed and marked *out of scope*.

Use it when:

- Understanding flow: *what feeds gold?*
- Spotting gaps: *bronze exists but no silver for this entity.*
- Navigating: click any card to open the **model inspector**.

The model inspector (side panel)

Opens from any model card, anywhere in the app. Six tabs:

Tab	Contents
Overview	Description, relation, partition/cluster config, upstream and downstream links, quick actions
Columns	Documented columns with name, `data_type`, description. Button to copy the whole contract
SQL	The raw Jinja you wrote, and the compiled GoogleSQL dbt generated
Tests	Every test attached, its type, column, and severity
Data	Live preview of the first 100 rows from BigQuery
Physical	Row count, size in bytes, partitioning, clustering, created and modified dates — read from the table definition. Warns if a large table has no partition column

---

Page 3 — Workbench

Purpose: query the warehouse **through dbt**, not through BigQuery directly.

This is the single most useful page for analysts. You write:

```
select
    company_code,
    period_month,
    sum(debit_amount)  as debit,
    sum(credit_amount) as credit,
    count(*)           as entries

from {{ ref('silver_gl_entries') }}

group by 1, 2
order by 1, 2
```

You never type a dataset name, and a typo in a model name is caught before a single byte is scanned.

> **Switching environment requires reloading the project.** `ref()` is resolved from `manifest.json`, and dbt freezes the physical dataset into that file when it parses. Changing the dropdown alone would leave every reference pointing at the previous environment. The UI therefore re-parses automatically when you switch, and the Overview page shows a red banner if the two ever drift apart.

Keyboard:

Shortcut	Action
`Ctrl+Enter`	Run — executes and returns rows (costs bytes)
`Ctrl+Shift+Enter`	**Validate — free.** BigQuery plans the query and returns the exact output columns and types without executing anything. Zero bytes billed.
`Ctrl+Space`	Column autocomplete. Reads the `ref()`s in your SQL and suggests their real column names with types
Type 2+ characters	Autocomplete for model and source names
`Tab`	Inserts two spaces

Result tabs:

Tab	Contents
Results	The grid, with type badges per column. Export as CSV or copy
Columns & types	The `name` + `data_type` contract for the result, ready to paste into a schema file
Compiled SQL	Exactly what was sent to BigQuery after `ref()` resolution
Lineage	Which relation each `ref()` resolved to

Validate is the feature to build a habit around. It tells you the output schema and the bytes the query *would* scan, for free, before you spend anything.

Page 4 — Documentation

Purpose: generate the column contract and descriptions for any model or query.

On entry you choose an engine — **AI** or **Pattern**. Covered in detail in section 6.

Two input modes:

- **From a model** — reads the live table definition, so types are the real ones. The model must have been built at least once.
- **From a query** — dry-runs your SQL to read its output schema. Nothing executes, nothing is billed.

Options:

Option	Effect
Profile the data	One aggregate pass measuring nulls, cardinality, ranges. Makes descriptions and test suggestions evidence-based. Costs a scan
Suggest tests	Only proposes a test the profile actually justifies
Include descriptions	Turn off for a bare type contract
Send sample values to Gemini	**AI only, off by default.** See section 6

Output tabs: Contract (a table), `name + data_type` (the bare list), Full schema YAML (ready to commit), Markdown (for a PR or Confluence).

You can write the YAML straight into the project. An existing file is backed up to `.bak` first. **Then click Refresh manifest** so dbt picks it up.

Page 5 — Silver Advisor

Purpose: turn a bronze table into concrete silver work, backed by measurement.

The bronze-to-silver step is where most judgment lives. This page replaces a blank file with a reasoned starting point.

How it works:

1. Pick a bronze (or silver) model
2. It profiles every column in one pass: null rates, cardinality, ranges, blank strings, sign distribution, constant and unique detection
3. It infers the business key and **verifies it with a real** `GROUP BY` — it does not guess whether duplicates exist, it checks
4. It emits recommendations across nine categories, each carrying the measurement that triggered it and a confidence of high / medium / low

The nine categories:

Category	Example recommendation
Deduplication	"*15 keys repeat, 15 surplus rows — deduplicate with* `row_number()` "
Null handling	" `vendor_customer` *is 33% null — flag it rather than coalescing to a fake default*"
Type cast	" `amount_local` *is FLOAT64 — cast to NUMERIC so totals do not drift*"
Standardisation	" `currency` *is compared and joined on — upper-case it once here*"
Categorisation	" `document_type` *has 3 values — map to labels with an explicit* `else 'Unmapped'` "
Aggregation	"*This is the gold grain this table supports; sum these measures by these dimensions*"
Quality flag	"*Stamp* `_has_sign_conflict` *rather than silently picking a winner*"
Pruning	" `fiscal_year` *is constant across all rows — omit it downstream*"
Testing	" `gl_entry_key` *is unique today — lock it in with a* `unique` *test*"

Then:

- Uncheck anything you disagree with (or use *High confidence only*)
- Open the **Generated silver model** tab
- It produces runnable SQL with the evidence as comments
- Write it into the project, refresh the manifest, build it

The generated model is a first draft for review, not a finished model. It calls project macros, so dbt must compile it — you cannot paste it into the Workbench.

Page 6 — Run Console

Purpose: run dbt with live streaming output. Replaces the terminal.

Command	What it does
`build`	Seed, run, and test in dependency order. **The default for most work**
`run`	Models only
`test`	Tests only, against what is already built
`seed`	Load the CSVs in `seeds/`
`parse`	Rebuild the manifest. Same as Refresh manifest
`compile`	Render SQL without touching the warehouse
`debug`	Check profile, credentials, connection
`deps`	Install `packages.yml`
`docs`	Generate the browsable catalog and lineage site
`source freshness`	Check how stale sources are

`--select` and `--exclude` accept the same syntax as the CLI:

Selector	Meaning
`silver_gl_entries`	just that model
`silver_gl_entries+`	that model and everything downstream
`+silver_gl_entries`	that model and everything upstream
`tag:bronze`	every model tagged bronze
`path:models/silver`	everything in that folder

Output streams line by line, colour-coded. One run at a time, because dbt writes to a shared `target/` directory. Runs can be cancelled. Session history is kept, and a run started on one page stays visible from every other page.

Page 7 — Catalog

Purpose: find things, and see how they connect.

Models tab — searchable table of every model with layer, materialization, column count, test count, doc status, and quick actions (query it, document it, build it).

Lineage tab — a dependency graph laid out by medallion layer. Click a node to inspect it; hover to highlight its edges.

Sources tab — declared sources and their freshness config.

Warehouse tab — read-only browser of the datasets you are permitted to see. Useful for "*what exists in BigQuery that is not in dbt yet?*"

---

6. The documentation engines

> The Documentation page is now one unified screen — see section 13 for how Source and Descriptions are chosen. This section is the deep comparison of the two description engines (AI vs Pattern); a third choice, **None**, simply leaves descriptions blank for you to fill in.

Both produce **the same artifact** — a dbt schema YAML block with name, `data_type`, description, and justified tests. Only the prose differs, which is what makes them comparable.

	AI (Gemini)	Pattern (rules)
Written by	A language model	Deterministic name-matching
Needs	A free API key	Nothing
Network	Yes	No
Reproducible	No — wording varies per run	Yes — identical every run
Infers business meaning	Yes	Only what is hardcoded
Recognises SAP field names	Yes, and reliably	Only the ones in its table
Coverage	Every column	Only what its rules match
Failure mode	**Confidently wrong**	Honest `TODO`
Flags uncertainty as	`Unclear: …`	`TODO …`

Real comparison

Both engines, same ad-hoc query, actual captured output:

```
select company_code,
       date_trunc(posting_date, month)      as period_month,
       sum(amount_local)                    as total_amount,
       count(distinct vendor_customer)      as counterparties,
       countif(debit_credit = 'C')          as credit_lines
from {{ ref('bronze_gl_entries') }}
group by 1, 2
```

Pattern engine — 3 of 5 described, 2 honest TODOs:

```
- name: company_code
  data_type: int64
  description: Company code identifying the legal entity.
- name: period_month
  data_type: date
  description: First day of the reporting month.
- name: total_amount
  data_type: numeric
  description: Monetary amount.
- name: counterparties
  data_type: int64
  description: TODO describe counterparties. Numeric measure.
- name: credit_lines
  data_type: int64
  description: TODO describe credit lines. Numeric measure.
```

AI engine — 5 of 5 described, much richer:

> `company_code` — Identifies the specific operating subsidiary or legal entity within the group responsible for the aggregated monthly metrics (SAP BUKRS).
>
> `total_amount` — The total aggregated transaction value in Indonesian Rupiah (IDR) accumulated by the entity during the month.
>
> `credit_lines` — The total count of active credit facilities or loan arrangements utilized by the company during the given month.

That last one is wrong. `credit_lines` is `countif(debit_credit = 'C')` — the number of accounting lines flagged as credit postings. It has nothing to do with credit facilities or loans. The model invented a plausible business meaning and stated it confidently, without an `Unclear:` prefix.

This is the tradeoff. The AI covers every column and writes better prose. It is also occasionally, invisibly wrong. The pattern engine never lies to you, it just leaves gaps.

Practical advice: use AI for the first pass, then read every line. It is much faster to correct one wrong description than to write seventeen from scratch.

Where the AI genuinely excels

On a well-named table it reads the profile and produces material a rules engine never could. Actual output for `bronze_gl_entries`:

> **Table:** Raw landing zone containing unverified general ledger entries directly ingested from source accounting systems. Serves as an immutable audit layer preserving historical financial transactions prior to downstream cleaning and aggregation.
>
> `fiscal_year` — Financial year in which the transaction is officially posted (SAP GJAHR), **currently fixed to 2026.**
>
> `currency` — Three-character transaction currency code, **currently populated exclusively with IDR.**
>
> `vendor_customer` — Identifier for the vendor or customer subledger account, **populated for two-thirds of entries.**

The bold parts come from the profile, not the column name. It correctly identified nine SAP field codes (BELNR, BUKRS, GJAHR, BUDAT, BLART, HKONT, KOSTL, SHKZG, SGTXT, DMBTR) and wrote a table-level summary. Cost: **one request, 2,148 tokens in, 492 out.**

Model options

All on the free tier. Because every column of a table goes in a **single batched request**, one table costs one request — so even the smallest quota is generous.

Model	Free quota	Notes
Gemini 3.6 Flash *(default)*	500/day, 15/min	Current generation. Available to all API keys
Gemini 2.5 Flash	250/day, 10/min	Older. **Not available to newly created keys**
Gemini 2.5 Pro	100/day, 5/min	Best at unfamiliar schemas
Gemini 2.5 Flash-Lite	1,000/day, 15/min	Fastest, shortest output. Good quota fallback

If a quota is hit, the UI says so and offers a one-click retry on Flash-Lite. The Pattern engine keeps working regardless.

Setup, and why it is free

Get a key at aistudio.google.com/apikey. **A personal Google account is fine.** No credit card, no GCP admin.

The Gemini key has nothing to do with your BigQuery project:

	BigQuery	Gemini
Purpose	Reads your warehouse	Writes descriptions
Auth	Your work `gcloud` login	A separate API key
Project	`data-analytics-asg`	Irrelevant

Free tier means free. There is no billing attached, so nothing can be charged. When a quota runs out the request simply fails.

> **Why not Vertex AI through our own GCP project?** It was tested and returns `PERMISSION_DENIED` for `aiplatform.endpoints.predict` on `data-analytics-asg` — it needs an admin grant. It also bills per token. The free key avoids both.

The key is stored in `dbt_ui/.runtime/ai.json`, which is gitignored, and is never returned to the browser except as a masked prefix. `GEMINI_API_KEY` in the environment overrides it.

Privacy: what leaves your machine

This matters, so it is explicit.

Always sent to Google (structure only): column names, data types, medallion layer, row count, null percentages, distinct counts, and the boolean flags `is_unique` / `is_constant` / `all_null`.

Only sent if you enable "Send sample values" (off by default): observed min/max per column, and the most frequent values.

That second group is **real data from your tables.** For `document_type` it is harmless (`SA` , `KR` , `DR`). For `amount_local` it is actual figures from your ledger. Leave it off unless you have decided that is acceptable for the table in front of you.

---

7. What is inside the generated documentation

The four output formats

1. Bare contract — the `name` + `data_type` list, nothing else. Hand this to whoever is building the next layer:

```
- name: gl_entry_key
  data_type: string
- name: document_number
  data_type: int64
- name: posting_date
  data_type: date
- name: amount_local
  data_type: numeric
- name: _bronze_loaded_at
  data_type: timestamp
```

2. Full schema YAML — commit-ready, with descriptions and tests:

```yaml
version: 2

models:
  - name: bronze_gl_entries
    description: >
      Raw landing zone for general ledger entries. One row in, one row out - no
      filtering, no deduplication, no business logic.
    config:
      materialized: table
    columns:
      - name: gl_entry_key
        data_type: string
        description: >
          Unique 32-character primary hash key generated to uniquely identify
          each general ledger line item.
        data_tests:
          - unique
          - not_null

      - name: document_type
        data_type: string
        description: >
          Two-character code classifying the type of accounting document
          (SAP BLART).
        data_tests:
          - not_null
          - accepted_values:
              arguments:
                values: [DR, KR, SA]
```

3. Markdown table — for a pull request or Confluence:

Column	Type	Null %	Distinct	Description
`gl_entry_key`	string	0%	15	Deterministic surrogate key…
`amount_local`	numeric	0%	15	Signed amount in local currency…

4. On-screen contract table — sortable, with profile columns when profiling is on.

Field by field

Field	Where it comes from	Notes
`name`	The warehouse	Nested `STRUCT` fields appear as dotted paths, e.g. `address.city`
`data_type`	The warehouse	Lower-cased GoogleSQL spelling
`description`	AI, existing YAML, or pattern rules	Precedence below
`data_tests`	The profile	Only tests the data justifies
`config.materialized`	The manifest	`table`, `view`, `incremental`

Description precedence. The UI labels which one won:

1. **AI** — if the AI engine ran
2. **existing** — a description already committed in the project YAML
3. **pattern** — matched a naming rule
4. **fallback** — nothing matched, emitted as `TODO describe …`

This is why running the Pattern engine on `bronze_gl_entries` showed all 17 columns as `existing`: the project already documents them, and committed prose is never silently overwritten.

Data types you will see

Read from BigQuery and normalised to GoogleSQL spelling. **BigQuery's REST API still returns legacy names; the UI converts them**, so you get the spelling you would actually write in DDL:

Legacy (raw API)	What the UI emits
`INTEGER`	`int64`
`FLOAT`	`float64`
`BOOLEAN`	`bool`
`RECORD`	`struct`
`NUMERIC`	`numeric`
`DATE` / `TIMESTAMP`	`date` / `timestamp`
any `REPEATED` field	`array<type>`

If you ever see `integer` in a YAML file, it was hand-written, not generated here.

Which tests get suggested, and why

Never a blanket set — each is earned by a measurement:

Test	Condition
`not_null`	The column had **zero** nulls in the profile
`unique`	Every value distinct **and** the name looks like a key (`*_key` , `*_id` , `*_number`)
`accepted_values`	Text column, 10 or fewer distinct values, under 50% of rows. Observed values are filled in for you

> `accepted_values` on a numeric column needs `quote: false` , or BigQuery rejects it with "*No matching signature for operator IN*". The generator adds it automatically. This was a real bug found by running the pipeline.

Profile statistics

With *Profile the data* on, one aggregate pass produces per column:

Statistic	Use
`null_count` / `null_pct`	Drives null-handling advice and `not_null`
`distinct_count` / `distinct_pct`	Identifies keys and code lists
`min` / `max`	Value range; character length for text
`blank_count`	Empty strings, which behave differently from NULL
`negative_count`	Mixed-sign measures needing a debit/credit split
`is_unique` , `is_constant` , `is_all_null`	Key detection and pruning

Above 50,000 rows it samples and says so. Percentages are estimates at that point, not a census.

---

8. Guardrails

Deliberate limits. Worth knowing before you hit one.

Dataset scope

The UI may only read the bronze and silver layers — in **every** environment, not just the selected one:

```
bronze_dbt        silver_dbt          (production)
dbt_dev_bronze    dbt_dev_silver      (dev sandbox)
dbt_ci_bronze     dbt_ci_silver       (CI sandbox)
```

The list covers all environments deliberately. The restriction is about *layers*, not environments, and scoping it to the selected target caused a false refusal: with the project loaded for dev and the dropdown on prod, `ref()` pointed at `dbt_dev_silver` while the allowlist only held `silver_dbt`, so a legitimate query was rejected. Widening it keeps the layer boundary exactly as strict while removing a failure that had nothing to do with the boundary.

Everything else is refused — gold, seeds, and all 46 other datasets including `GOLD`, `SILVER`, `ASG_DATALAKE`, and every `VS_*` set.

Enforced by **two independent checks**:

1. **Syntactic**, before any network call. Every dataset your SQL names must be allowed. Catches `select * from gold_dbt.x`. Costs nothing — the statement never leaves your machine.
2. **Semantic**, via a free dry run. Every *physical* table BigQuery would read must be allowed. Catches a permitted view that selects from a forbidden dataset — the text never names it, but the data would still reach your screen.

Out-of-scope models still appear in listings, dimmed and marked, because hiding them would make the lineage look wrong. The warehouse browser lists only permitted datasets, so it cannot be used to enumerate the rest of the project.

Change the boundary with `DBT_UI_ALLOWED_DATASETS=a,b,c`, then restart.

Build scope — gold is never written from the UI

The dataset allowlist governs **reads**, and it cannot police dbt itself: `dbt build` runs as a subprocess issuing its own SQL, which never passes through the guard. So there is a second, independent control.

Every dbt command the UI issues has `--exclude tag:gold` appended, unconditionally, merged with any exclusion you type. There is no code path in the UI that builds gold. Verified: a full `dbt build` from the Run Console reports `PASS=41` rather than `PASS=46`, the gold model and its four tests simply absent, and the string `dbt_dev_gold` never appears in the log.

Selectors that explicitly target gold are refused with a 403 rather than silently resolving to nothing:

You ask for	Result
`--select tag:gold`	refused
`--select path:models/gold`	refused
`--select gold_gl_monthly_summary`	refused, by tag lookup in the manifest
`--select gold_gl_monthly_summary+`	refused
`--select tag:bronze`	allowed, with `--exclude tag:gold` still appended

In the UI this shows up as a **read-only** badge on the Gold column of the Pipeline board with its build button removed, no *Build this model* action in the inspector for a gold model, and a notice in the Run Console.

`deps`, `debug` and `clean` are untouched, since they do not select nodes.

Change with `DBT_UI_BLOCKED_LAYERS=gold,something_else`, or set it empty to disable the restriction.

> **The orchestrator is deliberately unaffected.** Production still needs gold built, and that happens from the command line, not from here.

What these guardrails are and are not

> **They prevent accidents, not determined access.** Your gcloud credentials still hold whatever BigQuery grants your account has. Anyone with a terminal can run `bq query` or `dbt build --select tag:gold` directly and bypass all of this. For a boundary that cannot be bypassed you need IAM — ask your GCP admin to scope the account's dataset grants.
>
> **The seed remains buildable.** `gl_entries` lands in `dbt_dev_seeds`, which is outside the read scope, but bronze depends on it. Blocking it would break the pipeline, so it is built but not readable from the UI.

Read-only workbench

`INSERT`, `UPDATE`, `DELETE`, `MERGE`, `CREATE`, `DROP`, `ALTER`, `TRUNCATE`, `GRANT` and friends are refused. Changes to data or schema belong in a model or seed so they go through review and land in the DAG. Comments and CTEs are stripped before the check, so a legitimate query starting with a comment works.

Spend cap

Every query runs with `maximum_bytes_billed` at 20 GiB. BigQuery **refuses the job** rather than running it, so a careless `select *` on a huge table costs nothing. Raise it with `DBT_UI_MAX_BYTES_BILLED`.

Row cap

Previews are wrapped in a subquery with a `LIMIT`, default 200. Wrapping rather than appending, so it cannot break a query that has its own `ORDER BY` or `LIMIT`.

File writes

Confined to the project directory, restricted to `.sql .yml .yaml .md .csv`, and blocked inside `target/`, `dbt_packages/`, `logs/` and `.git/`. Overwrites leave a `.bak`.

dbt commands

An allow-list — the browser sends a verb, never a command line. Selector strings are validated before being passed as argv, so `a; drop table b` is refused.

Authentication and roles

The UI now **requires sign-in**, and what each user can do is governed by a role (Admin / Manager / Analyst). This is enforced in the backend, not just the UI. See section 11 for the model and section 12 for managing it.

The server still binds to `127.0.0.1` and the session cookie travels over plain HTTP, so **do not bind it to** `0.0.0.0` without TLS in front. The role system guards against accidents among trusted teammates; it is not a substitute for BigQuery IAM, which is what actually limits what the underlying credentials can reach.

---

9. Troubleshooting

Changes to the code do not appear

Almost always multiple servers running. The newest could not bind the port, moved to another, and your browser is talking to the old one.

```
Get-Process python          # more than one? that's the problem
taskkill /F /IM python.exe
python dbt_ui\serve.py
```

Then `Ctrl+F5`.

"Reauthentication is needed"

```
gcloud auth application-default login
```

Then **restart the server** — it caches the BigQuery client at startup.

"No target/manifest.json"

Click **Refresh manifest**, or run `dbt parse --profiles-dir .`.

Edited a file but the UI shows the old version

Click **Refresh manifest**. The UI reads `target/manifest.json`, which only updates when dbt parses.

"was not found in location"

A region mismatch. Every production dataset in `data-analytics-asg` is in `asia-southeast2`, and BigQuery cannot join across regions. Check the target's `location` in `profiles.yml`.

> The legacy `data_analytics_asg_test` profile is pinned to `US` and only works for self-contained seed runs. Prefer `--target test` on the main profile.

"That model is not available to this key"

Google retired an older Gemini model for new keys. Pick **Gemini 3.6 Flash**.

"400 INVALID_ARGUMENT" from Gemini

A parameter the chosen model does not support. Already handled for the shipped models; if a new one misbehaves, switch models.

"Free-tier quota reached"

Wait for the daily reset, click the one-click retry on Flash-Lite, or use the Pattern engine.

"has no relation yet. Build it first"

Types are read from the live table, so the model must exist. Build it once.

A model shows "out of scope"

It lives outside the permitted datasets. Expected for gold, seeds and the example models. See section 8.

---

10. Reference

Commands

```
# UI
python dbt_ui\serve.py                    # start
python dbt_ui\serve.py --check            # environment report, then exit
python dbt_ui\serve.py --port 9000        # different port
python dbt_ui\serve.py --no-browser       # do not open a browser

# dbt (all need --profiles-dir . from the project root)
dbt deps --profiles-dir .
dbt debug --profiles-dir .
dbt parse --profiles-dir .
dbt build --profiles-dir . --target dev
dbt build --profiles-dir . --target dev --select silver_gl_entries+
dbt test  --profiles-dir . --select tag:silver
dbt docs generate --profiles-dir . --static
```

Environment variables

Variable	Default	Effect
`DBT_UI_HOST`	`127.0.0.1`	Interface to bind
`DBT_UI_PORT`	`8777`	Port; next free one used if taken
`DBT_UI_ALLOWED_DATASETS`	bronze + silver	Comma-separated dataset allowlist (reads)
`DBT_UI_BLOCKED_LAYERS`	`gold`	Layers the UI may never build. Empty disables it
`DBT_UI_MAX_BYTES_BILLED`	`21474836480`	Per-query spend cap in bytes
`DBT_UI_VERBOSE`	unset	Log every HTTP request
`GEMINI_API_KEY`	unset	Overrides the stored key

Backend modules

File	Responsibility
`config.py`	Project discovery, targets, layers, dataset allowlist
`manifest.py`	Reader over `target/manifest.json`
`warehouse.py`	BigQuery via the dbt profile; dry-run type introspection; scope enforcement
`sql_scope.py`	Extracts table references from SQL for the syntactic guard
`jinja_sql.py`	`ref()` / `source()` resolution, read-only policy
`typing_map.py`	`INTEGER` → `int64` and friends
`profiling.py`	Single-pass column profiling
`recommend.py`	Silver Advisor rules
`codegen.py`	Schema YAML, pattern documentation, silver generation
`ai_docs.py`	Gemini documentation, key storage, quota mapping
`runner.py`	dbt subprocesses, job registry, log buffers
`api.py`	JSON routes
`server.py`	stdlib HTTP server

Dependencies

Nothing beyond what dbt already installs, except `google-genai` for AI documentation. No Node, no build step, no bundler. Verified on dbt-core 1.12.3, dbt-bigquery 1.12.0, Python 3.14.

Glossary

Term	Meaning
model	One `.sql` file containing a `SELECT`. Becomes one table or view
seed	A CSV in `seeds/`, version-controlled, loaded by `dbt seed`
source	A table dbt reads but does not create, declared in YAML
materialization	How a model is built: `table`, `view`, `incremental`
target	A named connection profile: `dev`, `test`, `prod`
manifest	`target/manifest.json` — dbt's compiled description of the project
DAG	The dependency graph dbt derives from your `ref()` calls
relation	A fully qualified `project.dataset.table`
grain	What one row of a table represents
dry run	BigQuery plans a query without executing it. Free, and returns the output schema

---

11. Signing in and roles (RBAC)

dbt Studio now requires you to sign in, and what you can do depends on your role. This is real access control: every permission is checked again on the server before an action runs, so hiding a button is a convenience, not the boundary — calling the API directly still returns `403`.

Signing in

Open the URL and you get a login screen. Enter your email and password. The session is kept in an `HttpOnly` cookie (12-hour life, 4-hour idle timeout), so page JavaScript can never read the token. Sign out from the identity box at the bottom of the sidebar.

The three roles

The names are a little counter-intuitive, so read this carefully: **Manager is the most powerful role, not Admin.**

Role	In one line
Manager	The privileged role. Modifies tables, runs dbt, manages users, roles and dataset access, configures the connection.
Admin	Full **visibility**, no changes. Sees every screen including configuration, but cannot write, run dbt, or manage users.
Analyst	Read-only. Views datasets, schemas, documentation, and queries data. No writes of any kind.

The permission matrix

Permission	Admin	Manager	Analyst
Login	✅	✅	✅
View Data Studio	✅	✅	✅
View tables	✅	✅	✅
Read data	✅	✅	✅
View database configuration	✅	✅	❌
Modify tables (write files)	❌	✅	❌
Manage user access	❌	✅	❌
Modify user roles	❌	✅	❌
Configure database	❌	✅	❌
Modify datasets	❌	✅	❌
Write/delete data (run dbt)	❌	✅	❌

This matrix is **editable** by a Manager — see section 12. Login is pinned on for every role and cannot be turned off.

Default accounts

Created automatically on first start, for development and testing only:

Email	Password	Role
`manager@gmail.com`	`manager123`	Manager
`admin@gmail.com`	`admin123`	Admin
`analyst@gmail.com`	`analyst123`	Analyst

Change these passwords once you are in (Settings → Your account, or a Manager can reset them). They are documented here, so treat them as public.

How it is stored

Users, password hashes, sessions and per-user dataset grants live in a small SQLite database at `dbt_ui/.runtime/studio.db` (gitignored). Passwords are hashed with PBKDF2-HMAC-SHA256 (per-user salt, high iteration count) — the plaintext is never stored. Only the SHA-256 of a session token is kept, so a database dump yields no usable sessions. A role change takes effect on that user's **next request**, even if they are already signed in.

---

12. The Settings page

A new page in the sidebar (⚙). What you see depends on your role — most of it is Manager-only.

BigQuery dataset access

Every dataset your credentials can see, each with a checkbox for whether this UI may use it. Tick to allow, untick to revoke. The saved list replaces the built-in default, and it drives the whole app — the workbench, autocomplete, documentation, everything.

- **Managers** can edit it. Everyone else sees it read-only.
- Ticking a box never grants access you do not already have in BigQuery IAM — it only decides what this app is *willing* to touch.
- Saved to `dbt_ui/.runtime/access.json`. An empty selection falls back to the built-in default (bronze + silver across every target).
- Per-user grants are also possible from the Users panel: a user can be restricted to a subset of the allowed datasets. A grant can only ever narrow the project list, never widen it.

Users & roles (Manager only)

A table of every registered user with their email, role, dataset access and status. A Manager can:

- **Change a role** — a dropdown per user; saves immediately and applies on that user's next request.
- **Add a user** — email, password, role.
- **Enable / disable** an account (disabling revokes their live sessions).
- **Reset a password**, or restrict a user to specific datasets.

Safety rails: you cannot demote or disable **your own** account, and the **last remaining Manager** cannot be demoted, disabled or deleted — otherwise nobody could ever manage roles again.

The permission matrix (editable)

The matrix from section 11 is shown here, and to a **Manager every cell is a clickable toggle**. Click a cell to flip that permission on or off for that role. It saves immediately, persists across restarts, and applies to everyone with that role on their next request. It is the real control — enabling "Modify tables" for Admin genuinely lets an admin write files, and turning it off restores the `403`.

Guardrails: **Login** is pinned on and shows a lock. You cannot remove "Modify user roles" from the last role that has it. Non-managers see the matrix read-only.

Your account

Change your own password (requires the current one).

---

13. The unified Documentation page

The Documentation page was rebuilt. Previously there were separate "engine" cards that were really the same machine relabelled. Now it is **one screen with two independent choices**:

Source — where the columns come from

Source	What it does	Output
A dbt model	Reads the live table definition of a model you pick	`models:` schema YAML
A query	Dry-runs a `SELECT` and uses its output columns	`models:` schema YAML
An existing table	A `dataset.table` dbt does **not** build (with autocomplete)	`sources:` block

Descriptions — who writes the prose

Engine	Notes
Pattern	Deterministic local rules. Free, offline, reproducible.
AI (Gemini)	Richer prose, understands SAP field names. Needs a free key.
None	Schema only — columns come back blank for you to fill in by hand.

The two are orthogonal: any source pairs with any engine. **dbt itself never writes descriptions** — it has no engine for that — so the prose always comes from Pattern or AI. See section 6 for the AI-vs-Pattern comparison, which still applies.

The proposal

After generating, you get an editable proposal:

- **Click any description to edit it in place.** The YAML rebuilds as you type.
- Tabs for the Contract, the bare `name + data_type`, the full YAML, and Markdown.
- A **download icon** (↓) next to Save opens a menu: YAML, CSV, Markdown, JSON. CSV is the one to hand to a spreadsheet.
- **Save** writes it into the project (backing up any existing file to `.bak`). Then Refresh manifest so dbt picks it up.

> Saving and running are gated by role. An Analyst can generate and download a draft (a read) but cannot Save (a write) — that is a Manager action.

---

14. Creating views and tables from the Workbench

The Workbench is still read-only for exploration, with **two sanctioned exceptions**: you can now create a **view** or a **table**, the way you would in the BigQuery console.

CREATE VIEW / CREATE TABLE by hand

Type a `CREATE VIEW …`, `CREATE OR REPLACE VIEW …`, or `CREATE [OR REPLACE] TABLE …` statement and run it (`Ctrl+Enter`). It executes and you get a "View created" / "Table created" confirmation instead of an empty grid.

Still blocked, deliberately: `CREATE FUNCTION`, `CREATE PROCEDURE`, `DROP`, `DELETE`, `INSERT`, `MERGE`, `TRUNCATE`, `ALTER`. Those change or destroy data and belong in a reviewed model.

The Create-table dialog

The ⊞ **Create table** button opens a BigQuery-console-style dialog:

- **Source** — an empty table, or the query currently in the editor (CTAS).
- **Destination** — project, dataset, table (project and dataset prefilled from the target).
- **Schema** — field rows (name / type / mode) with an "edit as text" toggle, for an empty table.
- **Partitioning** — none, or partition by a column.
- A live SQL preview, then Create.

There is also a ◫ **Create view** button that wraps the current SELECT in a `CREATE OR REPLACE VIEW … AS`.

Table autocomplete everywhere it helps

Both the Workbench editor and the source-declaration input suggest **datasets** first, then **tables** inside a dataset after you type the dot. So you never have to remember table names — type `bronze_dbt.` and pick from the list.

> Creating a view/table is a **write**, so it is a Manager action (`403` for Analyst/Admin). It writes to BigQuery within the allowed dataset scope only, and objects created this way are **not** part of the dbt DAG — for anything you want dbt to manage, use a model or declare a source (section 15).

---

15. Documenting tables dbt did not build

dbt only documents what is in the project. To make dbt aware of a **pre-existing / foreign table** (one it did not build — e.g. a Debezium landing table), you declare it as a **dbt source**. The Documentation page does this when you set **Source = An existing table**:

1. Type the table (`dataset.table`, with autocomplete).
2. It reads the real schema from BigQuery (a free metadata call) and drafts a `sources:` block, with Pattern or AI descriptions.
3. Edit the descriptions in place, then **Save** to `models/_sources.yml`.
4. **Register with dbt** — a one-click step that runs `dbt parse` then `dbt docs generate`, so dbt now recognises the table, includes it in the docs site and lineage, and lets you reference it with `source('…', '…')`.

The Silver Advisor was also extended: it can now profile and recommend silver work for **any** in-scope table, not just dbt-built ones. A foreign table can even generate a silver model — it just reads the table by its full name instead of `ref()`, with a note in the SQL on how to promote it to a source.

> Reading the schema and drafting descriptions is free (metadata + local rules or the Gemini free tier). "Register with dbt" runs dbt commands (Manager only) and `dbt docs generate` touches the warehouse catalog, so it needs BigQuery access.

---

16. What it costs

The honest headline: **dbt Studio, dbt Core, and this UI are all free.** Every real cost is BigQuery (and, if you host the UI, a little compute). These figures come from Google's published on-demand pricing and reasonable assumptions about a 100-table medallion project — **not** a measured bill.

The pricing that matters

Meter	Rate
BigQuery compute (on-demand)	$6.25 / TiB scanned. **First 1 TiB/month free.**
BigQuery storage (active)	~$0.023 / GiB / month. **First 10 GiB free.**
dbt Core	$0 (open source)
dbt Studio (this UI)	$0 (Python stdlib, runs locally)

The 10 MB minimum. BigQuery bills a **minimum of 10 MB per query, and 10 MB per table a query references**. Cleansing 1 KB costs the same as 10 MB. For many small tables the *number of queries* drives cost more than data volume. Free things worth remembering: failed and cached queries, `CREATE VIEW` (no scan), and batch loads/copies/exports/deletes all cost **$0**.

Daily transformation of 100 tables (Debezium not counted)

Assuming the pipeline runs once a day and the source data is already in BigQuery:

Approach	Compute/month	Notes
Incremental (recommended)	**$0**	Only the daily delta is scanned; ~29–293 GiB/month stays under the 1 TiB free tier
Full-refresh, small tables	**$0**	~59 GiB/month, still free
Full-refresh, ~1 GiB tables each	~$30	Full scan × 30 days; avoid — use incremental

Storage: silver is a view ($0); bronze + gold as tables ≈ **$1–2/month** for tens of GiB. So the daily transform is realistically **~$0–3/month**.

Deploying to GCP for a team of 8 (2–3 hrs/day, daily)

If you host the UI on GCP instead of a laptop, and **BigQuery query cost is counted separately** (as you asked), the only new cost is running the app:

Component	$/month
Cloud Run service (UI, ~3 hrs/day active, scale-to-zero)	~$3–6
Cloud Scheduler (daily dbt trigger)	$0 (free tier)
Artifact Registry (container image)	~$0.10
Secret Manager / networking (same region)	~$0
Total hosting	**≈ $3–6 / month**

So **deploying dbt Studio for 8 people costs roughly $3–6/month** in GCP compute, on top of whatever BigQuery bills for the queries themselves.

If you include the interactive query cost

Eight analysts running light exploration (documentation, previews — this UI caps every query at 20 GiB and previews are LIMIT-wrapped) typically stay **inside the 1 TiB/month free tier**, so query cost is often **$0**. Heavy multi-GB scans across the team could push it to tens of dollars, but that is a workload choice, not a floor.

Real-time (CDC) instead of daily

Cost scales almost linearly with how often you transform to keep data fresh:

Freshness	Transform compute/month*
Daily (batch)	$0
Hourly	~$2–3
Every 15 min	~$30
Every 5 min	~$100 → **switch to slot/Editions pricing here**

* incremental, delta hitting the 10 MB floor per table. At high frequency the per-query floor dominates; that is the point where BigQuery **capacity (slot) pricing** — which has no 10 MB floor — becomes cheaper than on-demand.

Bottom line

- **Software: $0.** dbt, dbt Core, this UI.
- **Daily 100-table transform: ~$0–3/month** (use incremental to stay at $0).
- **Hosting for 8 people on GCP: ~$3–6/month.**
- **Total realistic monthly cost: roughly $5–15**, dominated by hosting, not by BigQuery, as long as tables are not enormous and you do not full-refresh them daily.

How to keep it cheap

1. **Incremental models** for large tables — the single biggest lever; keeps compute at $0.
2. **Partition + cluster** by date so queries scan less.
3. **Run only as often as the data needs** — daily beats hourly 24×.
4. **Watch tests** — each `dbt test` is its own query with its own 10 MB floor; 100 models × 4 tests = 400 queries. Keep tests meaningful.
5. The 20 GiB per-query cap is already on, so a runaway scan fails rather than bills.

> Every number here is from Google's price list and assumed workload sizes, not your actual usage. The two things that move it most are **table size** and **incremental vs full-refresh**. Nothing has been measured against the project yet because the BigQuery `jobs.create` permission is still unresolved.